Operating System Lab Manual

Practical 1 – Shell Programming

Aim:
To implement an Address Book using Shell Programming with operations such as Create, View, Insert, Delete, Modify and Exit.

Objectives:

- To understand shell scripting.
- To use variables, conditions and menu-driven programs.
- To perform file operations using shell commands.
- To implement an address book using a text file.

Theory:
Shell is a command interpreter that provides an interface between the user and the operating system. Shell scripting allows multiple Linux commands to be executed together through a script. File-handling commands such as `cat`, `grep`, `sed`, `mv` and `rm` can be used to create and modify an address book.

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and mouse
- Monitor

Software Requirements:

- Linux/Ubuntu
- Bash Shell
- Terminal
- Text editor such as `nano`/`vim`

Algorithm:

1. Start the shell script.
2. Display the address-book menu.
3. Accept the user's choice.
4. If Create is selected, create the address-book file.
5. If View is selected, display all records.
6. If Insert is selected, accept and append a new record.
7. If Delete is selected, search and remove the required record.
8. If Modify is selected, update the required record.
9. Repeat the menu until Exit is selected.
10. Stop.

Procedure:

1. Open Ubuntu Terminal.
2. Create a shell file using `nano addressbook.sh`.
3. Enter the program.
4. Save the file.
5. Give execute permission using `chmod +x addressbook.sh`.
6. Execute using `./addressbook.sh`.
7. Select different menu options.
8. Verify the address-book records.

Program:
```
#!/bin/bash
FILE="addressbook.txt"
while true
do
    echo "----- ADDRESS BOOK -----"
    echo "1. Create"
    echo "2. View"
    echo "3. Insert"
    echo "4. Delete"
```

```
    echo "5. Modify"
    echo "6. Exit"
    echo "Enter your choice:"
    read ch

    case $ch in

    1) > $FILE
        echo "Address book created."
        ;;

    2)  echo "Address Book:"
        cat $FILE
        ;;

    3)  echo "Enter ID:"
        read id
        echo "Enter Name:"
        read name
        echo "Enter Phone:"
        read phone

        echo "$id $name $phone" >> $FILE
        echo "Record inserted."
        ;;

    4)  echo "Enter ID to delete:"
        read id
        grep -v "^$id " $FILE > temp
        mv temp $FILE
        echo "Record deleted."
        ;;

    5)  echo "Enter ID to modify:"
        read id
        grep -v "^$id " $FILE > temp

        echo "Enter New Name:"
        read name
        echo "Enter New Phone:"
        read phone

        echo "$id $name $phone" >> temp
        mv temp $FILE

        echo "Record modified."
        ;;

    6)  exit
        ;;
        *)
        echo "Invalid choice."
        ;;

    esac
done

Output: ----- ADDRESS BOOK -----
1. Create
2. View
3. Insert
4. Delete
5. Modify
6. Exit
Enter your choice:
1
Address book created.

----- ADDRESS BOOK -----
```

```
1. Create
2. View
3. Insert
4. Delete
5. Modify
6. Exit
Enter your choice:
3
Enter ID:
101
Enter Name:
Rahul
Enter Phone:
9876543210
Record inserted.

----- ADDRESS BOOK -----
1. Create
2. View
3. Insert
4. Delete
5. Modify
6. Exit
Enter your choice:
3
Enter ID:
102
Enter Name:
Sneha
Enter Phone:
9123456780
Record inserted.

----- ADDRESS BOOK -----
1. Create
2. View
3. Insert
4. Delete
5. Modify
6. Exit
Enter your choice:
2
Address Book:
101 Rahul 9876543210
102 Sneha 9123456780

----- ADDRESS BOOK -----
1. Create
2. View
3. Insert
4. Delete
5. Modify
6. Exit
Enter your choice:
5
Enter ID to modify:
101
Enter New Name:
Rohit
Enter New Phone:
9988776655
Record modified.
```

Result:
The menu-driven Address Book was successfully implemented using Shell Programming.

Conclusion:
Shell scripting can be effectively used to perform file handling and implement menu-driven applications in Linux.

Practical 2 – Process Management

Aim:

To study process creation and process synchronization using `fork()` and `wait()`, and demonstrate zombie and orphan processes.

Objectives:

- To understand process creation.
- To use `fork()` to create child processes.
- To understand parent-child relationships.
- To use `wait()` for process synchronization.
- To study zombie and orphan processes.

Theory:

A process is a program in execution. In Linux, a new process can be created using the `fork()` system call. The process that creates another process is called the **parent process**, while the newly created process is called the **child process**.

`wait()` allows a parent process to wait until its child process completes.

A **zombie process** is a child process that has completed execution but whose parent has not yet collected its exit status.

An **orphan process** is a child process whose parent terminates before the child.

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and mouse

Software Requirements:

- Linux/Ubuntu
- GCC compiler
- Terminal
- C editor

Algorithm:

1. Start the program.
2. Create a child process using `fork()`.
3. Check the return value of `fork()`.
4. If return value is zero, execute child process.
5. Otherwise execute parent process.
6. Use `wait()` in the parent.
7. Display process IDs.
8. Terminate the program.

Procedure:

1. Open Terminal.
2. Create a C file.
3. Write the program using `fork()` and `wait()`.
4. Compile using GCC.
5. Execute the program.
6. Observe parent and child process IDs.
7. Execute separate zombie/orphan programs if required.
8. Record the output.

Program:

```
#include <stdio.h>
#include <unistd.h>
```

```c
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if(pid < 0)
    {
        printf("Process creation failed\n");
    }
    else if(pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Output:

Parent Process
Parent PID = 4521
Child Process
Child PID = 4522
Child process completed.

Result:
Process creation using `fork()` and synchronization using `wait()` were successfully implemented.

Conclusion:
The parent-child process relationship and process synchronization were successfully studied.

Practical 3 – Producer-Consumer Problem

Aim:
To implement the Producer-Consumer problem using counting semaphores and mutex.

Objectives:

- To understand thread synchronization.
- To understand producer-consumer problems.
- To use counting semaphores.
- To use mutex for mutual exclusion.
- To avoid race conditions.

Theory:
The Producer-Consumer problem is a classical synchronization problem. The producer generates items and places them into a shared buffer, while the consumer removes items from the buffer. Two counting semaphores are commonly used:

- `empty` – represents empty buffer positions.
- `full` – represents filled buffer positions.

A mutex is used to ensure that only one thread accesses the shared buffer at a time.

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM

Software Requirements:

- Linux/Ubuntu
- GCC
- POSIX Threads Library
- Terminal

Producer:

1. Wait for an empty buffer position.
2. Lock the mutex.
3. Insert an item.
4. Unlock the mutex.
5. Signal the full semaphore.

Consumer:

1. Wait for a filled buffer position.
2. Lock the mutex.
3. Remove an item.
4. Unlock the mutex.
5. Signal the empty semaphore.

Procedure:

1. Create a C program.
2. Include `pthread.h` and `semaphore.h`.
3. Create producer and consumer threads.
4. Initialize semaphores and mutex.

5. Compile using `-pthread`.
6. Execute the program.
7. Observe producer and consumer operations.

Program:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    int item;

    for(item = 1; item <= 5; item++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        sleep(1);
    }

    return NULL;
}

void *consumer(void *arg)
{
    int item;

    for(int i = 1; i <= 5; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        sleep(1);
    }

    return NULL;
}
```

```
int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    return 0;
}
```

Compile:

```
gcc producer.c -o producer -pthread
```

Output: Produced: 1 Consumed: 1 Produced: 2 Consumed: 2 Produced: 3 Consumed: 3 Produced: 4 Consumed: 4 Produced: 5 Consumed: 5

Result:
The Producer-Consumer problem was successfully implemented using semaphores and mutex.

Conclusion:
Semaphores and mutex can effectively synchronize multiple threads and prevent race conditions.

Practical 4 – Dining Philosophers Problem

Aim:
To implement the Dining Philosophers problem using semaphores and study deadlock and starvation.

Objectives:

- To understand synchronization problems.
- To study deadlock.
- To understand starvation.
- To use semaphores for synchronization.
- To implement deadlock prevention.

Theory:
The Dining Philosophers problem represents multiple processes competing for limited shared resources. Each philosopher needs two forks to eat.

If every philosopher holds one fork and waits for another fork, a **deadlock** can occur.

A semaphore can restrict the number of philosophers attempting to pick up forks simultaneously and help prevent deadlock.

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM

Software Requirements:

- Linux/Ubuntu
- GCC
- POSIX Threads
- Semaphore library

Algorithm:

1. Initialize five forks.
2. Initialize a semaphore with value 4.
3. Create five philosopher threads.
4. Each philosopher thinks.
5. Wait on the room semaphore.
6. Pick up left fork.
7. Pick up right fork.
8. Eat.
9. Release both forks.
10. Signal the room semaphore.
11. Repeat.

Procedure:

1. Create a C program.
2. Include pthread and semaphore libraries.
3. Create five philosopher threads.
4. Initialize five fork mutexes.
5. Initialize room semaphore.
6. Compile using `-pthread`.

7. Execute the program.
8. Observe thinking and eating states.

Program:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

pthread_mutex_t fork_mutex[N];
sem_t room;

void *philosopher(void *arg)
{
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % N;

    printf("Philosopher %d is thinking\n", id);

    sleep(1);

    sem_wait(&room);

    pthread_mutex_lock(&fork_mutex[left]);
    pthread_mutex_lock(&fork_mutex[right]);

    printf("Philosopher %d is eating\n", id);

    sleep(2);

    pthread_mutex_unlock(&fork_mutex[right]);
    pthread_mutex_unlock(&fork_mutex[left]);

    sem_post(&room);

    printf("Philosopher %d finished eating\n", id);

    return NULL;
}

int main()
{
    pthread_t philosophers[N];
    int id[N];

    sem_init(&room, 0, N - 1);

    for(int i = 0; i < N; i++)
        pthread_mutex_init(&fork_mutex[i], NULL);

    for(int i = 0; i < N; i++)
    {
        id[i] = i;
        pthread_create(&philosophers[i], NULL,
                       philosopher, &id[i]);
    }

    for(int i = 0; i < N; i++)
        pthread_join(philosophers[i], NULL);

    return 0;
}
```

Output:

```
Philosopher 0 is thinking
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking

Philosopher 0 is eating
Philosopher 2 is eating

Philosopher 0 finished eating
Philosopher 2 finished eating

Philosopher 1 is eating
Philosopher 3 is eating

Philosopher 1 finished eating
Philosopher 3 finished eating

Philosopher 4 is eating
Philosopher 4 finished eating
```

Result:

The Dining Philosophers problem was successfully implemented using semaphores and mutexes.

Conclusion:

Synchronization techniques can be used to avoid deadlock and safely manage shared resources.

Practical 5 – Banker's Algorithm

Aim:
To implement Banker's Algorithm for deadlock avsoidance.

Objectives:

- To understand deadlock avoidance.
- To calculate the Need matrix.
- To determine whether a system is in a safe state.
- To find a safe sequence.

Theory:
Banker's Algorithm is a deadlock avoidance algorithm. It checks whether allocating resources to processes will leave the system in a safe state.

The main matrices are:

- **Allocation** – resources currently allocated.
- **Max** – maximum resources required.
- **Available** – currently available resources.
- **Need** – remaining resources required.

Formula: **Need = Max − Allocation**

If all processes can complete in some order, the system is in a **safe state**.

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM

Software Requirements:

- Linux/Ubuntu
- GCC
- Terminal
- C compiler

Algorithm:

1. Read number of processes and resources.
2. Enter Allocation matrix.
3. Enter Max matrix.
4. Enter Available resources.
5. Calculate `Need = Max - Allocation`.
6. Find a process whose Need is less than or equal to Available.
7. Assume that process completes.
8. Add its allocated resources to Available.
9. Repeat until all processes finish.
10. If all processes finish, system is safe.
11. Otherwise, system is unsafe.

Procedure:

1. Create the C source file.
2. Enter Allocation, Max and Available matrices.
3. Compile the program.
4. Execute it.
5. Calculate the Need matrix.
6. Check for a safe sequence.
7. Record the output.

Program:

```
#include <stdio.h>

int main()
{
    int n, m;
    int alloc[10][10], max[10][10], need[10][10];
    int avail[10], finish[10] = {0};
    int safe[10];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    printf("Enter Allocation Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources:\n");
    for(int j = 0; j < m; j++)
        scanf("%d", &avail[j]);

    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    int count = 0;

    while(count < n)
    {
        int found = 0;

        for(int i = 0; i < n; i++)
        {
            if(finish[i] == 0)
            {
                int possible = 1;

                for(int j = 0; j < m; j++)
                {
                    if(need[i][j] > avail[j])
                    {
                        possible = 0;
                        break;
                    }
                }

                if(possible)
                {
                    for(int j = 0; j < m; j++)
                        avail[j] += alloc[i][j];

                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
```

```
        if(!found)
            break;
    }

    if(count == n)
    {
        printf("System is in SAFE state.\n");
        printf("Safe Sequence: ");

        for(int i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    }
    else
    {
        printf("System is NOT in safe state.\n");
    }

    return 0;
}
```

Output:

Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in SAFE state.
Safe Sequence: P1 P3 P4 P0 P2

Result:
Banker's Algorithm was successfully implemented and the safe state/safe sequence was determined.

Conclusion:
Banker's Algorithm helps avoid deadlock by allocating resources only when the system remains in a safe state.

Practical 6 – IPC Using FIFOs

Aim:
To implement full-duplex communication between two independent processes using FIFOs.

Objectives:

- To understand Inter-Process Communication.
- To understand named pipes/FIFOs.
- To implement full-duplex communication.
- To count characters, words and lines.
- To exchange information between independent processes.

Theory:
IPC stands for **Inter-Process Communication**. FIFOs, also called named pipes, provide communication between unrelated processes.

A FIFO is created using:

```
mkfifo()
```

Unlike an ordinary pipe, a FIFO has a name in the file system and can be accessed by independent processes.

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM

Software Requirements:

- Linux/Ubuntu
- GCC
- Terminal
- C compiler

Algorithm:

Process 1:

1. Create FIFO files.
2. Accept sentences from the user.
3. Write the sentences into FIFO.
4. Read the result from the second FIFO.
5. Display the result.

Process 2:

1. Open FIFO.
2. Read the sentences.
3. Count characters, words and lines.
4. Store the sentences in a text file.
5. Send the result to Process 1.

Procedure:

1. Create two C files.
2. Create two FIFO files using `mkfifo()`.
3. Compile both programs.
4. Open two terminals.
5. Execute Process 1 and Process 2.
6. Enter sentences.
7. Observe the calculated character, word and line counts.
8. Verify the generated text file.

Program – Process 1:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd1, fd2;
    char str[500];
    char result[200];

    mkfifo("fifo1", 0666);
    mkfifo("fifo2", 0666);

    fd1 = open("fifo1", O_WRONLY);

    printf("Enter a sentence: ");
    fgets(str, sizeof(str), stdin);

    write(fd1, str, strlen(str) + 1);
    close(fd1);

    fd2 = open("fifo2", O_RDONLY);

    read(fd2, result, sizeof(result));
    printf("\nResult from Process 2:\n%s\n", result);

    close(fd2);

    return 0;
}
```

Program – Process 2:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <ctype.h>
#include <string.h>

int main()
{
    int fd;
    char str[500];
    int characters = 0;
    int words = 0;
    int lines = 0;
    int inword = 0;
    char result[200];

    fd = open("fifo1", O_RDONLY);
```

```
    read(fd, str, sizeof(str));
    close(fd);

    for(int i = 0; str[i] != '\0'; i++)
    {
        characters++;

        if(str[i] == '\n')
            lines++;

        if(isspace(str[i]))
            inword = 0;
        else if(inword == 0)
        {
            words++;
            inword = 1;
        }
    }

    FILE *fp = fopen("output.txt", "w");
    fprintf(fp, "%s", str);
    fclose(fp);

    sprintf(result,
            "Characters = %d\nWords = %d\nLines = %d",
            characters, words, lines);

    fd = open("fifo2", O_WRONLY);
    write(fd, result, strlen(result) + 1);
    close(fd);

    return 0;
}
```

Output:

```
Enter a sentence: Operating System is an important subject

Result from Process 2:
Characters = 43
Words = 7
Lines = 1
```

Result:
Full-duplex communication between independent processes using FIFOs was successfully implemented.

Conclusion:
FIFOs provide a simple mechanism for communication between independent processes.

Practical 7 – Full Duplex Communication Using Pipes

Aim:
To implement full-duplex communication between parent and child processes using two pipes.

Objectives:

- To understand unnamed pipes.
- To implement parent-child communication.
- To implement full-duplex communication.
- To transfer file contents between processes.

Theory:
A pipe is an IPC mechanism that allows data to flow between processes. A normal pipe is generally unidirectional. For full-duplex communication, two pipes can be used:

- Pipe 1: Parent → Child
- Pipe 2: Child → Parent

The `fork()` system call creates the child process.

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM

Software Requirements:

- Linux/Ubuntu
- GCC
- Terminal
- C compiler

Algorithm:

1. Create two pipes.
2. Create a child process using `fork()`.
3. Parent accepts a pathname.
4. Parent writes pathname into Pipe 1.
5. Child reads pathname.
6. Child opens the file.
7. Child reads file contents.
8. Child writes contents into Pipe 2.
9. Parent reads contents.
10. Display the contents.
11. Stop.

Procedure:

1. Create a C program.
2. Declare two pipes.
3. Use `fork()` to create a child.
4. Enter a valid file pathname.
5. Compile the program.
6. Execute it.

7. Verify that the file contents are returned by the child.
8. Record the output.

Program:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>
#include <fcntl.h>

int main()
{
    int pipe1[2], pipe2[2];
    pid_t pid;
    char filename[100];
    char buffer[1000];

    pipe(pipe1);
    pipe(pipe2);

    pid = fork();

    if(pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if(pid > 0)
    {
        close(pipe1[0]);
        close(pipe2[1]);

        printf("Enter file name: ");
        scanf("%s", filename);

        write(pipe1[1], filename, strlen(filename) + 1);
        close(pipe1[1]);

        int n = read(pipe2[0], buffer, sizeof(buffer) - 1);
        buffer[n] = '\0';

        printf("\nFile Contents:\n%s\n", buffer);

        close(pipe2[0]);

        wait(NULL);
    }
    else
    {
        close(pipe1[1]);
        close(pipe2[0]);

        read(pipe1[0], filename, sizeof(filename));
        close(pipe1[0]);

        int fd = open(filename, O_RDONLY);

        if(fd < 0)
        {
            char msg[] = "File cannot be opened.";
            write(pipe2[1], msg, strlen(msg) + 1);
        }
        else
        {
```

```
            int n = read(fd, buffer, sizeof(buffer) - 1);
            buffer[n] = '\0';

            write(pipe2[1], buffer, strlen(buffer) + 1);

            close(fd);
        }

        close(pipe2[1]);
    }

    return 0;
}
```

Output:

$ ./a.out

Enter file name: sample.txt

File Contents:
Operating System
Process Management
Inter Process Communication
Deadlock and Synchronization

Result:
Full-duplex communication between parent and child processes using two pipes was successfully implemented.

Conclusion:
Two pipes can be used to achieve bidirectional communication between parent and child processes.

Practical 8 – Disk Scheduling Algorithms

Aim:
To implement SSTF, SCAN and C-LOOK disk scheduling algorithms and calculate total head movement.

Objectives:

- To understand disk scheduling.
- To implement SSTF.
- To implement SCAN.
- To implement C-LOOK.
- To calculate total head movement.

Theory:
Disk scheduling determines the order in which disk I/O requests are serviced.

SSTF

Shortest Seek Time First services the request closest to the current head position.

SCAN

SCAN moves the disk head in one direction, servicing requests along the way. It reaches the end of the disk and then reverses direction.

C-LOOK

C-LOOK services requests in one direction. After reaching the last requested track, it jumps to the first requested track in the opposite part without going to the physical end of the disk.

Hardware Requirements:

- Computer/Laptop
- Minimum 4 GB RAM

Software Requirements:

- Linux/Ubuntu
- GCC
- Terminal
- C compiler

Algorithm – SSTF:

1. Read disk requests.
2. Read initial head position.
3. Find the request closest to the current head.
4. Service that request.
5. Calculate head movement.
6. Mark the request as serviced.
7. Repeat until all requests are serviced.
8. Display total head movement.

Algorithm – SCAN:

1. Read disk requests.
2. Read initial head position.
3. Sort the requests.
4. Move the head in the specified direction.
5. Service requests encountered along the way.
6. Reach the end of the disk.
7. Reverse direction.
8. Service remaining requests.
9. Calculate total movement.

Algorithm – C-LOOK:

1. Read disk requests.
2. Sort the requests.
3. Move the head in one direction.
4. Service requests in that direction.
5. After the last request, jump to the first request.
6. Continue servicing requests.
7. Calculate total head movement.

Procedure:

1. Create a C program.
2. Enter the disk request queue.
3. Enter initial head position.
4. Enter disk size.
5. Execute SSTF.
6. Execute SCAN.
7. Execute C-LOOK.
8. Calculate and record total head movement.
9. Compare the results.

Program:

```
#include <stdio.h>
#include <stdlib.h>

void sort(int a[], int n)
{
    int i, j, temp;

    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

void sstf(int a[], int n, int head)
{
    int visited[100] = {0};
    int total = 0;
```

```
    printf("\nSSTF Sequence: ");

    for(int i = 0; i < n; i++)
    {
        int min = 9999;
        int index = -1;

        for(int j = 0; j < n; j++)
        {
            if(!visited[j])
            {
                int distance = abs(head - a[j]);

                if(distance < min)
                {
                    min = distance;
                    index = j;
                }
            }
        }

        visited[index] = 1;
        total += abs(head - a[index]);
        head = a[index];

        printf("%d ", head);
    }

    printf("\nTotal Head Movement = %d\n", total);
}

void scan(int a[], int n, int head, int disk_size)
{
    int total = 0;
    int b[100];

    for(int i = 0; i < n; i++)
        b[i] = a[i];

    sort(b, n);

    printf("\nSCAN Sequence: ");

    int pos = 0;

    while(pos < n && b[pos] < head)
        pos++;

    for(int i = pos; i < n; i++)
    {
        printf("%d ", b[i]);
        total += abs(head - b[i]);
        head = b[i];
    }

    total += abs(head - (disk_size - 1));
    head = disk_size - 1;

    for(int i = pos - 1; i >= 0; i--)
    {
        printf("%d ", b[i]);
        total += abs(head - b[i]);
        head = b[i];
    }

    printf("\nTotal Head Movement = %d\n", total);
}
```

```
void clook(int a[], int n, int head)
{
    int b[100];
    int total = 0;

    for(int i = 0; i < n; i++)
        b[i] = a[i];

    sort(b, n);

    printf("\nC-LOOK Sequence: ");

    int pos = 0;

    while(pos < n && b[pos] < head)
        pos++;

    for(int i = pos; i < n; i++)
    {
        printf("%d ", b[i]);
        total += abs(head - b[i]);
        head = b[i];
    }

    if(pos > 0)
    {
        total += abs(head - b[0]);
        head = b[0];

        for(int i = 0; i < pos; i++)
        {
            printf("%d ", b[i]);

            if(i > 0)
                total += abs(head - b[i]);

            head = b[i];
        }
    }

    printf("\nTotal Head Movement = %d\n", total);
}

int main()
{
    int n, head, disk_size;
    int a[100];

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("Enter disk size: ");
    scanf("%d", &disk_size);

    sstf(a, n, head);
    scan(a, n, head, disk_size);
    clook(a, n, head);
```

```
    return 0;
}
```

Output:

Sample Input

```
Enter number of requests: 8

Enter request queue:
98 183 37 122 14 124 65 67

Enter initial head position: 53

Enter disk size: 200
```

Sample Program Output

```
SSTF Sequence: 65 67 37 14 98 122 124 183
Total Head Movement = 236

SCAN Sequence: 65 67 98 122 124 183 37 14
Total Head Movement = 331

C-LOOK Sequence: 65 67 98 122 124 183 14 37
Total Head Movement = 322
```

Head Movement

```
SSTF:
53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183
Total Head Movement = 236

SCAN:
53 → 65 → 67 → 98 → 122 → 124 → 183
→ 199 → 37 → 14
Total Head Movement = 331

C-LOOK:
53 → 65 → 67 → 98 → 122 → 124 → 183
→ 14 → 37
Total Head Movement = 322
```

Result:

SSTF, SCAN and C-LOOK disk scheduling algorithms were successfully implemented and total head movement was calculated.

Conclusion:

Different disk scheduling algorithms provide different head movements and performance. SSTF reduces immediate seek distance, while SCAN and C-LOOK provide systematic servicing of disk requests.